# MODULE 4

## Handling HTML forms with PHP

One of the most powerful features of PHP is the way it handles HTML forms. The basic concept that is important to understand is that any form element will automatically be available to your PHP scripts.

To create a form, you use the `<form>` element as follows:

```
<form action="form.php" method="post">
</form>
```

The <form> element has two important attributes:

- **action**: specifies the URL that processes the form submission. In this example, the form.php will process the form.
- **method**: specifies the HTTP method for submitting the form. The most commonly used form methods are POST and GET. In this example, the form method is post.

The form method is case-insensitive. It means that you can use either post or POST. If you don't specify the method attribute, the form element will use the get method by default.

Typically, a form has one or more input elements including text, password, checkbox, radio button, select, file upload, etc. The input elements are often called form fields.

An input element has the following important attributes name, type, and value. The name attribute will be used for accessing the value in PHP.

**POST method**

POST is a superglobal method, which collects form data and submits it to the HTTP server. The data entered is encoded, and the content is hidden. POST method has a global scope, and data is accessed from any script.

The POST method is preferred because data sent through it is not visible in the URL. The POST method is also important because data cannot be decoded by looking into web server logs.

POST does not have a limitation on the amount of data sent from the form. This is because data is submitted via the body of the HTTP request. The POST method is appropriate for a login form

If we specify the form method to be **POST**, then the form-data is sent to the server using the HTTP POST method.

```
<html>
    <body>

        <form action="form-handler.php" method="POST">
            Name: <input type="text" name="name"> <br/>
            Email: <input type="text" name="email"> <br/>
            <input type="submit">
        </form>

    </body>
</html>
```

In the code above, we have used the `<form>` tag to create an HTML form, with input fields for Name and Email along with submit button to submit the form-data.

Below, we have the code, to access the form-data in the Php file specified in the `action` attribute of our HTML form.

```php
<?php

// getting the value of name field
$name = $_POST["name"];
// getting the value of the email field
$email = $_POST["email"];

echo "Hi, ". $name . "<br>";
echo "Your email address: ". $email ."<br>";

?>
```

You will get the below output, if you provide name as "Studytonight" and email address as "we@studytonight.com".

OUTPUT:

```
Hi, Studytonight

Your email address: we@studytonight.com
```

**GET method**

GET is the default super global method that collects or retrieves data from the server. It has a global scope. Thus, data is accessed from any script in the program. The GET method submits data in the URL.

Data transferred via this method is visible on the URL of the HTTP request. The HTTP request can be cached and saved in the browser history. The disadvantage of the GET method is that it should not be used with sensitive data such as passwords because it is not secure.

The GET method has a limitation of the amount of data sent from the form. The data being sent on the URL depends on the web server's operating system and the type of browser.

Most systems have a limit of 255 characters. The best example of using the GET method is with the search engine forms. The code below is an example;

```
<html>
    <body>

        <form action="form-handler.php" method="GET">
            Name: <input type="text" name="name"> <br/>
            Email: <input type="text" name="email"> <br/>
            <input type="submit">
        </form>

    </body>
</html>
```

If we specify the form method to be GET, then the form-data is sent to the server using the HTTP GET method.

We have two input fields in above form, one is name and the other one is email. When we click on submit, we will be redirected to the following URL, *form-handler.php?name=Studytonight&email=name@studytonight.com*, with the form-data appended to the URL.

Below, we have the code, to access the form-data in the Php file specified in the action attribute of our HTML form, this time using the GET superglobal.

```php
<?php

// getting the value of name field
$name = $_GET["name"];
// getting the value of the email field
$email = $_GET["email"];

echo "Hi, ". $name . "<br>";
echo "Your email address: ". $email ."<br>";

?>
```

Again, the output remains the same.

OUTPUT:

```
Hi, Studytonight

Your email address: we@studytonight.com
```

**Tabular comparison of GET and POST methods**

| POST Method | GET Method |
|---|---|
| Bookmarking the results is not possible. | Results can be bookmarked. |
| Data do not remain in the browser history. It's hidden. | Data remain in browser history. |
| The performance is low because POST cannot decode the data. | The performance is high because of the simple nature of displaying data. |
| It is more secure | It is less secure |
| Do not limit the amount of data sent to the server. | Limit the amount of data sent to the server. |
| It works with sensitive data. | It cannot work with sensitive data. |

**PHP self-processing form**

Sometimes, you want to include both form and logic for handling form submission in a single PHP file. This form is often referred to as a self-processing form.

To create a self-processing form, you can use the `$_SERVER['REQUEST_METHOD']` that returns the request method e.g., `GET` or `POST`.

If the `$_SERVER['REQUEST_METHOD']` is GET, you show the form. And if the `$_SERVER['REQUEST_METHOD']` is POST, you process it. For example:

```
<!DOCTYPE html>
<html>
<head>
    <title>Self-Processing Form</title>
</head>
<body>
    <h1>Contact Form</h1>
    <?php
    // Check if the form is submitted
    if ($_SERVER["REQUEST_METHOD"] == "POST") {
        $name = $_POST["name"];
        $email = $_POST["email"];
        $message = $_POST["message"];

        // Process the form data (you can add more validation and processing here)
        // For this example, let's just display the submitted data
        echo "<p><strong>Name:</strong> $name</p>";
        echo "<p><strong>Email:</strong> $email</p>";
        echo "<p><strong>Message:</strong> $message</p>";
    }
    ?>

    <form method="post" action="<?php echo $_SERVER['PHP_SELF']; ?>">
        <label for="name">Name:</label>
        <input type="text" name="name" id="name" required><br>

        <label for="email">Email:</label>
        <input type="email" name="email" id="email" required><br>

        <label for="message">Message:</label><br>
        <textarea name="message" id="message" rows="4" cols="50" required></textarea><br>

        <input type="submit" value="Submit">
    </form>
</body>
</html>
```

In the above example:

- The form is displayed when the PHP script is loaded.
- When the form is submitted, the script checks if the request method is POST. If it is, the submitted data is processed and displayed.
- The `action` attribute of the form is set to `<?php echo $_SERVER['PHP_SELF']; ?>`, which means the form will be submitted to the same script.
- Remember that this is a basic example. In a real-world scenario, you would likely want to add more validation and security measures to handle user input safely.
- Also, note that in modern web development, using a PHP framework or combining PHP with JavaScript (AJAX) might provide a more dynamic and user-friendly experience for handling forms and interactions.

**Handling Radio Buttons**

Handling radio buttons in PHP forms is quite straightforward. Radio buttons are used when you want the user to select a single option from a set of choices. Here's how you can create and handle radio buttons in a PHP form:

```
<form method="post" action="process_form.php">
    <label>
        <input type="radio" name="gender" value="male"> Male
    </label>
    <label>
        <input type="radio" name="gender" value="female"> Female
    </label>
    <label>
        <input type="radio" name="gender" value="other"> Other
    </label>
    <input type="submit" value="Submit">
</form>
```

In this example, we have created a form with three radio buttons representing gender choices.

**PHP Form Handling (process_form.php):**

```php
<?php
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    if (isset($_POST["gender"])) {
        $selectedGender = $_POST["gender"];
        echo "You selected: " . $selectedGender;
    } else {
        echo "Please select a gender.";
    }
}
?>
```

- In this PHP script, we first check if the form has been submitted using the `$_SERVER["REQUEST_METHOD"]` variable.
- Then, we check if the `gender` field (the name of the radio buttons) is set in the `$_POST` array.
- If it is set, we retrieve the selected value and display it.
- If it's not set, we display a message indicating that a gender must be selected.

Remember to replace `"process_form.php"` with the actual filename where you are handling the form submission. Radio buttons are used to allow users to select a single option from a set of choices. In PHP, you can handle radio button selections by checking the `$_POST` array for the selected value when the form is submitted.

**Handling Select Options In PHP Forms**

Handling select options in PHP forms involves creating a form with a <select> element in HTML and then processing the selected option on the server-side using PHP. Here's a step-by-step guide:

The HTML Form:

```html
<!DOCTYPE html>
<html>
<head>
    <title>Select Option Form</title>
</head>
<body>
    <form method="post" action="process_form.php">
        <label for="color">Select your favorite color:</label>
        <select name="color" id="color">
            <option value="red">Red</option>
            <option value="green">Green</option>
            <option value="blue">Blue</option>
            <option value="yellow">Yellow</option>
        </select>
        <button type="submit">Submit</button>
    </form>
</body>
</html>
```

**The PHP Processing Script (`process_form.php`):**

```php
<?php
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    // Get the selected color from the form
    $selectedColor = $_POST["color"];

    // Process the selected color (you can perform any desired action here)
    echo "Your favorite color is: " . ucfirst($selectedColor);
}
?>
```

In this example, when the user selects a color and submits the form,

- the selected color is sent as a POST parameter to the `process_form.php` script.
- The PHP script retrieves the selected color using `$_POST["color"]`, processes it (in this case, capitalizing the first letter), and
- It then echoes a message back to the user.

**Handling Check Boxes In PHP Forms**

Handling checkboxes in PHP forms involves processing the user's input and dealing with the checked or unchecked state of checkboxes. Here's a step-by-step guide on how to handle checkboxes in PHP forms:

**HTML Form Markup:** Create an HTML form that includes checkboxes within a `<form>` element. Each checkbox should have a unique `name` attribute and a corresponding `value`.

```
<form method="post" action="process_form.php">
  <input type="checkbox" name="option1" value="1"> Option 1<br>
  <input type="checkbox" name="option2" value="1"> Option 2<br>
  <input type="checkbox" name="option3" value="1"> Option 3<br>
  <!-- Other form fields and submit button -->
</form>
```

**Processing Form Data (process_form.php):** Create a PHP script (e.g., `process_form.php`) that will handle the form submission. Use the `$_POST` superglobal to access the values submitted via the form.

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    // Check if each checkbox is checked
    $option1Checked = isset($_POST['option1']);
    $option2Checked = isset($_POST['option2']);
    $option3Checked = isset($_POST['option3']);

    // Process the checked options
    if ($option1Checked) {
        // Handle Option 1
    }
    if ($option2Checked) {
        // Handle Option 2
    }
    if ($option3Checked) {
        // Handle Option 3
    }

    // Perform other processing as needed

    // Redirect or display a success message
}
?>
```

In this example, each checkbox's `name` attribute corresponds to an element in the `$_POST` array. If a checkbox is checked, its value will be present in the `$_POST` array, and you can use the `isset()` function to check its state.

## FORM VALIDATION AND ERROR HANDLING

Form validation is a crucial step in web development to ensure that the data submitted through a form meets certain criteria before it's processed or stored in a database. This helps prevent malicious input, data errors, and other issues that might arise from incorrect or unwanted data.

Here's a basic example of how you can perform form validation in PHP:

1. **HTML Form** (index.html):

```html
<!DOCTYPE html>
<html>
<head>
    <title>Form Validation Example</title>
</head>
<body>
    <form action="process_form.php" method="post">
        <label for="name">Name:</label>
        <input type="text" id="name" name="name" required>
        <br>
        <label for="email">Email:</label>
        <input type="email" id="email" name="email" required>
        <br>
        <input type="submit" value="Submit">
    </form>
</body>
</html>
```

2. **PHP Validation** (process_form.php):

```php
<?php
// Initialize variables to store errors
$errors = array();

// Validate Name
if (empty($_POST['name'])) {
    $errors['name'] = "Name is required.";
}

// Validate Email
if (empty($_POST['email'])) {
    $errors['email'] = "Email is required.";
} elseif (!filter_var($_POST['email'], FILTER_VALIDATE_EMAIL)) {
    $errors['email'] = "Invalid email format.";
}

// Check for errors
if (count($errors) > 0) {
    // Display errors and redirect back to the form
    session_start();
    $_SESSION['errors'] = $errors;
    header("Location: index.html");
    exit();
} else {
    // Process the data (e.g., save to database)
    $name = $_POST['name'];
    $email = $_POST['email'];
    // ... Perform data processing here ...
    echo "Data submitted successfully!";
}
?>
```

In this example, the HTML form contains fields for the user's name and email. The required attribute ensures that these fields cannot be submitted empty. In the PHP script (process_form.php), the $_POST array is used to access the submitted data. The script validates the name and email fields, checking for empty values and ensuring the email has a valid format using FILTER_VALIDATE_EMAIL.

If any validation errors are found, they are stored in the $errors array. If no errors are found, the data can be further processed. If errors are detected, the user is redirected back to the form with an error message indicating the issues.

## SECURING USER INPUTS AND PREVENTING COMMON ATTACKS (XSS, CSRF)

Securing user inputs in PHP is crucial to prevent various types of attacks, such as SQL injection, cross-site scripting (XSS), and more. Here are some detailed examples of how you can secure user inputs in PHP:

1. **SQL Injection Prevention:** SQL injection occurs when malicious SQL code is injected into a query, potentially giving unauthorized access to the database. To prevent this, you should use prepared statements with parameterized queries.

```php
<?PHP
// Unsafe way (vulnerable to SQL injection)
$username = $_POST['username'];
$password = $_POST['password'];
$query = "SELECT * FROM users WHERE username = '$username' AND password = '$password'";

// Safe way using prepared statements
$stmt = $pdo->prepare("SELECT * FROM users WHERE username = :username AND password = :password");
$stmt->bindParam(':username', $username);
$stmt->bindParam(':password', $password);
$stmt->execute();
$user = $stmt->fetch();

?>
```

2. **Cross-Site Scripting (XSS) Prevention:** XSS occurs when malicious scripts are injected into web pages and executed in the context of a user's browser. To prevent this, you should sanitize and escape user inputs before displaying them in your HTML.

```php
<?PHP

// Unsafe way (vulnerable to XSS)
$name = $_GET['name'];
echo "Hello, $name!";

// Safe way using htmlspecialchars()
$name = $_GET['name'];
echo "Hello, " . htmlspecialchars($name) . "!";

?>
```

3. **Cross-Site Request Forgery (CSRF) Prevention:** CSRF attacks trick users into performing unwanted actions on a different site without their knowledge. To prevent this, use tokens to validate requests.

```php
<?PHP

// Generate and include a CSRF token in forms
$csrfToken = bin2hex(random_bytes(32));
$_SESSION['csrf_token'] = $csrfToken;
echo "<form action='process.php' method='post'>";
echo "<input type='hidden' name='csrf_token' value='$csrfToken'>";
// Other form fields
echo "</form>";

// Verify CSRF token in process.php
session_start();
if ($_POST['csrf_token'] === $_SESSION['csrf_token']) {
    // Process the form
} else {
    // CSRF token validation failed
}

?>
```

4. **File Upload Security:** When allowing users to upload files, ensure that you validate and sanitize the uploaded file to prevent any security risks.

```php
<?PHP

$targetDir = 'uploads/';
$targetFile = $targetDir . basename($_FILES['file']['name']);

// Validate and move the uploaded file
if (move_uploaded_file($_FILES['file']['tmp_name'], $targetFile)) {
    echo "File is valid and successfully uploaded.";
} else {
    echo "Upload failed.";
}

?>
```

5. **Input Validation and Filtering:** Always validate and sanitize user inputs for their intended use. For example, use `filter_var()` to validate email addresses or URLs.

```php
<?PHP

$email = $_POST['email'];
if (filter_var($email, FILTER_VALIDATE_EMAIL)) {
    // Valid email format
} else {
    // Invalid email format
}

?>
```

6. **Password Hashing:** When storing user passwords, never store them in plain text. Instead, use strong cryptographic hashing functions like password_hash() to securely store passwords

```php
$password = $_POST['password'];
$hashedPassword = password_hash($password, PASSWORD_DEFAULT);

// Store $hashedPassword in the database
```

7. **Input Filtering for Database Queries:** Apart from prepared statements, consider using input filtering functions like filter_input() to sanitize and validate user inputs before using them in database queries.

```php
$username = filter_input(INPUT_POST, 'username', FILTER_SANITIZE_STRING);
$password = filter_input(INPUT_POST, 'password', FILTER_SANITIZE_STRING);

// Proceed with the query using $username and $password
```

8. **Input Length and Format Validation:** Validate the length and format of user inputs to ensure they adhere to expected patterns.

```php
$phone = $_POST['phone'];
if (preg_match("/^[0-9]{10}$/", $phone)) {
    // Valid phone number format (10 digits)
} else {
    // Invalid phone number format
}
```

9. **Content Type Validation for File Uploads:** If you're allowing file uploads, validate the content type of uploaded files to ensure they match the expected types.

```php
$allowedTypes = ['image/jpeg', 'image/png'];
$uploadedFileType = $_FILES['file']['type'];

if (in_array($uploadedFileType, $allowedTypes)) {
    // Process the file upload
} else {
    // Invalid file type
}
```

10. **HTTP Only and Secure Cookies:** When dealing with cookies, set the "HttpOnly" and "Secure" flags to prevent client-side scripts from accessing them and ensure they are only transmitted over secure HTTPS connections.

```php
session_set_cookie_params([
    'httponly' => true,
    'secure' => true,
    'samesite' => 'Strict'
]);
```

Others PHP Security measures include:

1. **Input Whitelisting:** Consider using input whitelisting where you explicitly define allowed values or patterns for user inputs, rejecting anything that doesn't match the whitelist.
2. **User-Agent and IP Filtering:** For critical actions or sensitive data, consider implementing user-agent and IP filtering to restrict access to trusted devices and IP ranges.
3. **Rate Limiting and CAPTCHA:** Implement rate limiting to prevent brute-force attacks and use CAPTCHA challenges for user interactions that require additional validation.
4. **Server-Side Validation:** Always perform validation and sanitization on the server side, even if you have client-side validation in place. Client-side validation can be bypassed.
5. **OWASP Top Ten:** Familiarize yourself with the OWASP Top Ten list of web application security risks and make sure your code addresses these vulnerabilities appropriately.
6. **Security Audits:** Periodically conduct security audits and vulnerability assessments on your application to identify and mitigate potential weaknesses.
7. **Secure File Uploads:** If you're allowing file uploads, ensure uploaded files are stored outside the web root, validate file extensions, and use server-side mechanisms to prevent malicious file execution.

Remember that security is a comprehensive process that requires a multi-layered approach. Keep up with security news, follow best practices, and be prepared to adapt your code as new threats and security measures emerge.